

Hashing

Definition

Hashing

- ▶ Hashing is a technique used for performing insertion, deletions, and finds in **constant average time**. typically used for efficient data retrieval

Hash Table

- ▶ A hash table is a data structure that stores key-value pairs using a hash function
- ▶ A key is a value associated with each record.

Hash Functions

- ▶ A Hashing function is a key - to - address transformation, which acts upon a given key to compute the relative position of the key in an array.

A simple Hash function

$$\text{HASH (KEYVALUE)} = \text{KEYVALUE MOD TABLESIZE}$$

Example:

Hash (92)

$$\text{Hash (92)} = 92 \bmod 10 = 2$$

The key value 92 is placed in the relative location 2

Properties of good Hash function

A good Hash Function should

- ▶ Minimize collisions
- ▶ Be easy and quick to compute
- ▶ Distribute keys evenly in the hash table

Applications of Hash tables

- ▶ Database systems
- ▶ Symbol tables
- ▶ Network processing algorithms
- ▶ Browse caches

COLLISIONS

- ▶ A collision occurs when two keys hash to the same index.

Example : 37, 24 , 7

Index	Slot
0	
1	
2	37
3	
4	24

- 37 is placed in index 2
- 24 is placed in index 4
- Now inserting 7
- Hash (7) = $7 \bmod 5 = 2$
- 2 collides

Collision Resolution strategies

- ▶ The process of finding another position for the collide record is called Collision

Resolution strategy.

1. Open Addressing

- ▶ All elements stored in the hash table itself.
- ▶ If collision occurs, probe for next empty slot.

Techniques:

1. Linear Probing: $\text{index} = (\text{h}(\text{key}) + i) \% \text{table_size}$
2. Quadratic Probing: $\text{index} = (\text{h}(\text{key}) + i^2) \% \text{table_size}$
3. Double Hashing: Use second hash function: $\text{index} = (\text{h1}(\text{key}) + i * \text{h2}(\text{key})) \% \text{table_size}$

2. Chaining

- ▶ Each bucket stores a linked list (or another dynamic structure) of keys.
- ▶ Collided keys are stored in the list at that bucket.
- ▶ Simpler to implement; allows load factor > 1 .

Let us consider a simple hash function as
“key mod 7” and sequence of keys as

50, 700, 76, 85, 92, 73, 101.

0	
1	
2	
3	
4	
5	
6	

Initial Empty Table

0	
1	50
2	
3	
4	
5	
6	

Insert 50

0	700
1	50
2	
3	
4	
5	
6	76

Insert 700 and 76

0	700
1	50
2	
3	
4	
5	
6	76

Insert 85: Collision Occurs, add to chain

0	700
1	50
2	
3	
4	
5	
6	76

Insert 92 Collision Occurs, add to chain

0	700
1	50
2	
3	73
4	
5	
6	76

Insert 73 and 101

Open Addressing: LINEAR PROBING:

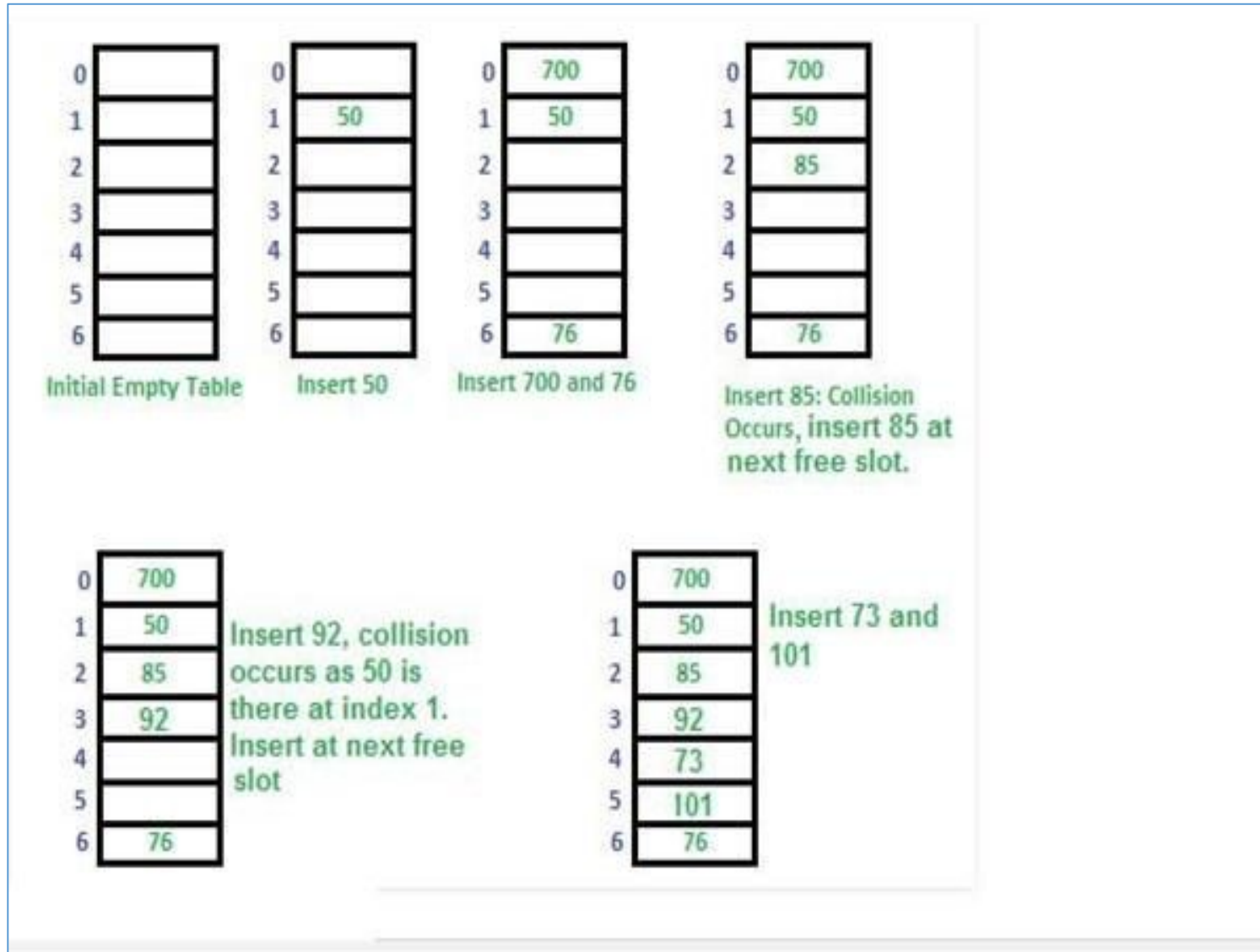
It is a collision handling method in Open Addressing.

In linear probing, we linearly probe for next slot.

Let S be the table size

- If slot $\text{hash}(x) \% S$ is full, then we try $(\text{hash}(x) + 1) \% S$
- If $(\text{hash}(x) + 1) \% S$ is also full, then we try $(\text{hash}(x) + 2) \% S$
- If $(\text{hash}(x) + 2) \% S$ is also full, then we try $(\text{hash}(x) + 3) \% S$

Let us consider a simple hash function as “key mod 7” and sequence of keys as 50, 700, 76, 85, 92, 73, 101.



QUADRATIC PROBING

- ▶ It is a collision handling method in open addressing.
- ▶ Quadratic probing is similar to linear probing and the only difference is the interval between successive probes or entry slots.
- ▶ If a collision occurs, the algorithm searches for the next available slot using a quadratic function of the probe number instead of linearly.

Formula:

$$\text{index} = (\text{h}(\text{key}) + i^2) \% \text{table_size}$$

Let us assume that the hashed index for an entry is index and at index there is an occupied slot. The probe sequence will be as follows:

$$\text{index} = \text{index} \% \text{hashTableSize}$$

$$\text{index} = (\text{index} + 1) \% \text{hashTableSize}$$

$$\text{index} = (\text{index} + 4) \% \text{hashTableSize}$$

$$\text{index} = (\text{index} + 9) \% \text{hashTableSize}$$

Example

Hash table size = 7

Hash function: $h(\text{key}) = \text{key} \% 7$

Insert keys: 10, 24, 31, 17

Key	$h(\text{key})$	Insert Position
10	3	3
24	3	4 ($3 + 1^2$)
31	3	0 ($3 + 2^2 = 7 \rightarrow 7 \% 7 = 0$)
17	3	2 ($3 + 3^2 = 12 \rightarrow 12 \% 7 = 5$? Wait...)

Index:	0	1	2	3	4	5	6
Keys:	31	-	-	10	24	17	-

Double Hashing

- ▶ Double hashing is a collision resolving technique in Open Addressed Hash tables. Double hashing uses the idea of applying a second hash function to key when a collision occurs.
- ▶ Double hashing can be done using :
$$(\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})) \% \text{TABLE_SIZE}$$
- ▶ Here $\text{hash1}()$ and $\text{hash2}()$ are hash functions and TABLE_SIZE is size of hash table. (We repeat by increasing i when collision occurs)

Load Factor (α)

- ▶ The load factor indicates how full the hash table is. The Load Factor (α) of a hash table is the ratio of the number of elements stored (n) to the size of the table (m): $\alpha = n/m$

Importance:

- ▶ High load factor $\rightarrow$ more collisions $\rightarrow$ performance drops.
- ▶ Low load factor $\rightarrow$ space underutilized.

Rule of thumb:

- ▶ $\alpha \leq 0.7$ for open addressing.
- ▶ α can be > 1 for chaining.

Real-world Use in Databases

- ▶ Hashing and hash tables are widely used in databases for efficient access and indexing.

Applications:

1. Indexing:

- ▶ Hash indexes map a search key to a data block location.
- ▶ Example: Quick lookup in key-value tables.

2. Caching:

- Store frequently accessed queries for $O(1)$ retrieval.

3. Uniqueness Checking:

- Hash table ensures primary key uniqueness during insert operations

4. Join Operations:

- ▶ Hash-based joins in relational databases for fast matching.

5. Data Deduplication:

- Use hash of records to identify duplicates efficiently.